Université des Sciences et de la Technologie Houari Boumediene Faculté d'Informatique

Master 1-IV, Année universitaire : **2025/2026**, Module : **Compilation**

Projet de réalisation d'un mini compilateur pour un mini Langage avec les Outils FLEX et BISON

I- Introduction

Ce projet a pour but de concevoir un mini-compilateur pour un langage nommé «**ProLang**», en intégrant les différentes étapes de la compilation : l'analyse lexicale, syntaxique, sémantique, la génération de code intermédiaire, optimisation du code intermédiaire, ainsi que la génération du code objet.

De plus, la gestion parallèle de la table des symboles et le traitement des différentes erreurs devront être pris en charge lors des phases d'analyse du processus de compilation.

II- Description du mini langage « ProLang»

II.1. Structure générale

La structure générale du programme est la suivante :

```
BeginProject  M1_IV_2526;
Setup :
        %% Partie déclarations
Run :
{
            //* Partie
                instruction   *//
}
EndProject ;
```

II.2. Partie déclaration

Cette section est dédiée à la déclaration des variables simples et constantes nécessaires au bon fonctionnement du programme.

1. Déclaration de variables

La syntaxe de déclaration d'une variable simple est la suivante :

define <nom_var> : <type> ;

Projet de réalisation d'un mini compilateur pour un mini Langage avec les Outils FLEX et BISON

⇨ La **déclaration** d'un **ensemble de variables** s'effectue en séparant les noms de variables par une barre verticale '|'.

Exemple :

%% Déclaration de 3 variables

define a | b | c: integer;

%% Avec initialisation

define b : integer = 10;

define moyenne : float = 0.0;

2. Déclaration d'un tableau

define <nom_tab> : [<type> ;<Taille>] **;**

⇨ La taille du tableau doit être une constante entière positive

Exemple :

define Tab : [integer ;20] **;**

3. Déclaration d'une constante

La syntaxe de déclaration d'une constante est la suivante :

const <nom_const> : <type>=Valeur ;

Exemple :

const Pi : float = 3.14159;

⇨ Valeur est une constante.

3. Identificateur

Un identificateur est une suite de lettres minuscules, de chiffres et/ou de tirets bas (« _ »), qui commence par une lettre alphabétique, ne dépasse pas 14 caractères, ne se termine pas par un tiret bas et ne contient pas de tirets bas consécutifs.

⇨ Les noms de programme, ainsi que les noms de variables et de constantes, sont des identificateurs.

Exemple: Isil_program, Myprogram, testprog, ….

Projet de réalisation d'un mini compilateur pour un mini Langage avec les Outils FLEX et BISON

4. Types

Le type peut être : integer, float.

Type	Description	Exemple
integer	Une constante entière est une suite de chiffres. Elle peut être signée ou non signée tel que sa valeur est entre **-32768** et **32767**. Si la constante entière est signée, elle doit être mise entre parenthèses.	**5, (-5), (+5)**
float	Une constante réelle est une suite de chiffres contenant le point décimal. Elle peut être signée ou non signée. Si la constante réelle est signée, elle doit être mise entre parenthèses.	**5.5, (-5.76), (+ 568.77)**

II.3. Partie Instruction

Les instructions autorisées dans notre langage sont :

Instruction	Description	Exemple
Affectation	Idf ← Expression ;	A ←23 ; B [5] ← (H+ 2) / (5 – b +(-3)) ;

Condition	**if** (condition) **then:**
	{
	%% instructions
	} **else** {
	%% instructions
	} **endIf;**
		if (Aa +2 > Bb) **then:**
		{
		H ← J – 5.5;
		} **else{** H←H-1 ;
		} **endIf ;**

Boucle	**loop while** (condition) { %% instructions } **endloop;**	**loop while** (x<10) { x ←x + 1; } **endloop;**

	for <idf> **in** <val_init> **to** <val_limit> **{**
	%% instructions
	} endfor;
		for i **in** 1 **to** 10 {
		Ka ← Hj * i;
		{
		endfor;

Lecture et écriture (input, output)	L'**output** permet d'afficher des informations à l'écran, tandis que l'**input** permet à l'utilisateur de saisir des données.	**in**(UserName) ; **out**("Vous avez saisi: ", UserName) ;

Projet de réalisation d'un mini compilateur pour un mini Langage avec les Outils FLEX et BISON

Important :

- Une condition est une expression qui renvoie '1' ou '0'. Elle peut prendre la forme d'une expression logique ou de comparaison.
- On peut avoir des **boucles** imbriquées.
- Toute instruction doit se terminer par un **point-virgule**.

Opérateurs

1. **Opérateurs arithmétiques :** +, -, * , /
2. **Opérateurs logiques**
   - **(**expression1 **AND** expression2**) :** le **et** logique**.**
   - **(**expression1 **OR** expression2**) :** le **ou** logique**.**
   - **NON(expression) :** la négation**.**
3. **Opérateurs de comparaison**
   - expression1> expression2
   - expression 1< expression2
   - expression1 >= expression2
   - expression1 <= expression2
   - expression1 == expression2
   - expression1 **!=** expression2

⇨ **Associativité et priorité des opérateurs**
⇨ **Associativité :** gauche pour tous les opérateurs.
⇨ **Priorité :** les priorités sont données par la table suivante par ordre croissant :

Opérateurs Logiques	AND
	OR
	NON
Opérateurs de comparaison	< > >= <= == **!=**
Opérateurs arithmétiques	+ -
	* /

Priorité ↓

Projet de réalisation d'un mini compilateur pour un mini Langage avec les Outils FLEX et BISON

II.4. Commentaires

Un commentaire peut être écrit sur une seule ligne sur une seule ligne en plaçant le texte après le symbole **%%** ou sur plusieurs lignes en le mettant entre //* et *// **.**

Exemple :

%% ceci est un commentaire sur une seule ligne

//* Ceci est un commentaire sur

Plusieurs lignes

*//

III- Travail à réaliser

Ci-dessous les différentes briques à implémenter afin de réaliser le compilateur demandé :

1. **Analyse lexicale avec l'outil FLEX**

Cette phase consiste à parcourir le programme source afin d'identifier les unités lexicales (tokens) telles que les mots-clés, identificateurs, constantes, opérateurs et séparateurs.

2. **Analyse syntaxico-sémantique avec l'outil BISON**

L'analyse syntaxique vérifie la conformité du programme par rapport à la grammaire du langage, tandis que l'analyse sémantique permet de détecter des erreurs liées au sens (types incompatibles, identificateurs non déclarés, etc.).

3. **Gestion de la table de symboles :**

La table de symboles est créée lors de l'analyse lexicale et regroupe les informations nécessaires sur les variables, constantes et structures du programme. Cette table doit être implémentée sous forme d'une **table de hachage** et doit être mise à jour tout au long du processus de compilation. Vous devez prévoir des procédures pour rechercher et insérer des éléments dans la table des symboles, et inclure également les variables structurées.

4. **Génération du code intermédiaire**

Le code intermédiaire doit être généré sous forme de quadruplets.

5. **Optimisation du code intermédiaire.**

Projet de réalisation d'un mini compilateur pour un mini Langage avec les Outils FLEX et BISON

6. **Génération du code machine.**

Le code machine doit être généré en assembleur 8086.

7. **Traitement des erreurs**

Le compilateur doit afficher des messages d'erreur clairs et précis, indiquant le type d'erreur, l'entité concernée, ainsi que la ligne et la colonne où elle est détectée.

Type_ de_ l'erreur, ligne 12, colonne 5, entité qui a générée l'erreur

BON COURAGE